Zephyr Project

IT-Schulungen.com
Technology & Transformation

C++ in Zephyr

C++-Anwendungscode sicher mit Zephyr-APIs verbinden

5.11

C++ beginnt mit drei klaren Entscheidungen

KONFIGURATION

C++ aktivieren

CONFIG_CPP=y bindet Laufzeitunterstützung und C++-Buildregeln ein.

QUELLCODE

Dateiendung entscheidet

.cpp und .cxx werden vom Buildsystem mit dem C++-Compiler übersetzt.

EINSTIEG

int main()

```
int main(void)
{
    return 0;
}
```

C++ ist Anwendungssprache in Zephyr – nicht Ersatz für Kernel-, Treiber- oder Systeminitialisierung.

Vom Quelltext zum Zephyr-Image

Fünf Stationen des gemischten Builds

01

Konfigurieren

CONFIG_CPP und Sprachstandard

02

Übersetzen

C und C++ getrennt kompilieren

03

Verbinden

C-Symbole eindeutig verlinken

04

Laufzeit

Konstruktoren und ABI bereitstellen

05

Image prüfen

Größe und Abhängigkeiten bewerten

Die Sprachgrenze liegt im Build und im ABI – nicht in getrennten Zephyr-Kerneln.

C und C++ sauber verbinden

C-SCHNITTSTELLE

```
extern "C" int sensor_read(struct sample *out);
```

Header	gemeinsame Typen und Funktionen in C- und C++-kompatibler Form deklarieren
Linkage	extern "C" verhindert C++-Namensmangling an der C-Grenze
Zephyr-APIs	C-APIs direkt aus C++ aufrufen und Rückgabewerte unverändert behandeln
Eigene Wrapper	C-Schnittstelle stabil halten; C++-Typen auf der Anwendungsseite kapseln

Die C-Grenze bleibt schmal und stabil; C++ organisiert Besitz, Typen und Lebensdauer dahinter.

RAII für Zephyr-Ressourcen

1 · **Besitz festlegen**	ein Wrapper besitzt die Ressource oder referenziert sie nur
2 · **Initialisieren**	Konstruktor oder Factory stellt einen gültigen Zustand her
3 · **Nutzen**	Methoden kapseln Zephyr-Aufrufe und Fehlercodes
4 · **Kopieren regeln**	Copy verbieten oder Besitzübertragung ausdrücklich definieren
5 · **Freigeben**	Destruktor räumt lokale Ressourcen deterministisch auf

RAII ist besonders wertvoll, wenn Besitz eindeutig und die Lebensdauer an einen Scope gebunden ist.

Typsicherheit ohne Laufzeitkosten

01	`enum class`	verhindert implizite Vermischung fachlich verschiedener Zustände
02	`constexpr`	verschiebt Konstanten, Tabellen und Prüfungen in die Übersetzungszeit
03	`Templates`	erzeugen typsichere statische Abstraktionen ohne virtuelle Dispatchkosten

C++ ist im Embedded-System besonders stark, wenn Abstraktion zur Compilezeit verschwindet.

Statische Objekte brauchen klare Grenzen

LEBENSZYKLUS

```
1  Treiberinitialisierung
2  globale C++-Konstruktoren
3  main()
4  keine globale Destruktion
```

01 Konstruktion

Globale Konstruktoren laufen nach der Treiberinitialisierung und vor main().

02 Abhängigkeiten

Konstruktoren dürfen keine noch nicht verfügbaren Dienste voraussetzen.

03 Destruktion

Globale Destruktion wird nicht unterstützt.

Bevorzugt lokale Objekte oder explizite Initialisierung, wenn Reihenfolge und Fehlerbehandlung relevant sind.

Bibliothek bewusst wählen

ZWEI BIBLIOTHEKSMODELLE

```
minimal: ABI-Grundlagen, new/delete, Konstruktoren
standard: STL und zusätzliche Laufzeitabhängigkeiten
```

01 Minimal C++

Grundlegende Sprachunterstützung ohne STL-Container und -Algorithmen.

02 Standardbibliothek

Erfordert passende Toolchain und C-Bibliothek; Umfang gezielt auswählen.

std::thread und std::mutex ersetzen Zephyrs eigene Thread- und Synchronisations-APIs nicht.

Laufzeitfeatures nach Kosten auswählen

EXCEPTIONS

Explizit aktivieren

CONFIG_CPP_EXCEPTIONS bringt zusätzliche Laufzeit- und Größeneffekte.

RTTI UND VIRTUAL

Gezielt einsetzen

Typinformationen und dynamischer Dispatch kosten Daten und Code.

HEAP

Bewusst begrenzen

new/delete benötigen eine passende Speicherstrategie und Fehlerbehandlung.

Jedes Laufzeitfeature braucht einen nachvollziehbaren Nutzen und ein gemessenes Ressourcenbudget.

C++-Leitlinien für Zephyr

ENTWURFSPRINZIP

```
C-API stabil halten
Besitz explizit modellieren
Compilezeit-Abstraktionen bevorzugen
Laufzeitkosten messen
```

01 Architektur

C++ im Anwendungscode; Zephyr-Ressourcen hinter kleinen Wrappern kapseln.

02 Projektpolicy

Sprachstandard, Bibliothek, Exceptions, RTTI und Heap bewusst festlegen.

Gutes Embedded C++ macht Lebensdauer und Typen klarer, ohne Ressourcenfolgen zu verstecken.